Lab - Deploying a MySQL Database Container
To Begin with the Lab

Summary

This lab demonstrates how to deploy a **MySQL database** inside an **Azure Container Instance (ACI)**. Initially, the deployment from Docker Hub fails due to Docker Hub's anonymous pull rate limit. As a workaround, the MySQL image is first pulled to the local machine, pushed to **Azure Container Registry (ACR)**, and then deployed successfully from ACR. Finally, the MySQL container is accessed to create a database, user, tables, and insert data.

Understanding

A **MySQL container** packages the MySQL database engine with all its required dependencies into a Docker image. Instead of installing MySQL on a virtual machine, you can run it directly as a container. Azure Container Instances provides the compute infrastructure automatically, so you only need to provide the container image and configuration.

Docker Hub hosts official MySQL images, but anonymous pull requests are subject to rate limits. To avoid this issue, the image can be downloaded locally, tagged, pushed to Azure Container Registry, and then deployed from ACR. MySQL uses environment variables (such as MYSQL_ROOT_PASSWORD) during the first startup to initialize the database securely.

Example:

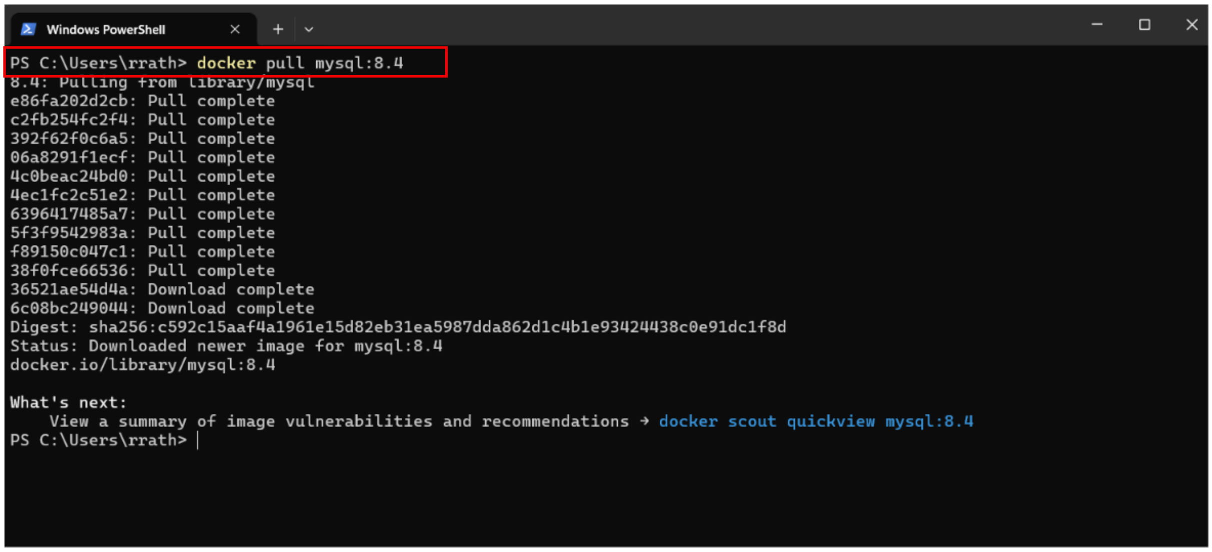
- Without Docker → Install MySQL manually on a VM.
- With Docker → Run MySQL inside a container using the official MySQL image.
- With Azure Container Instances → Azure manages the infrastructure while MySQL runs inside the container.

Lab Steps

Step 1: Pull MySQL Image Locally

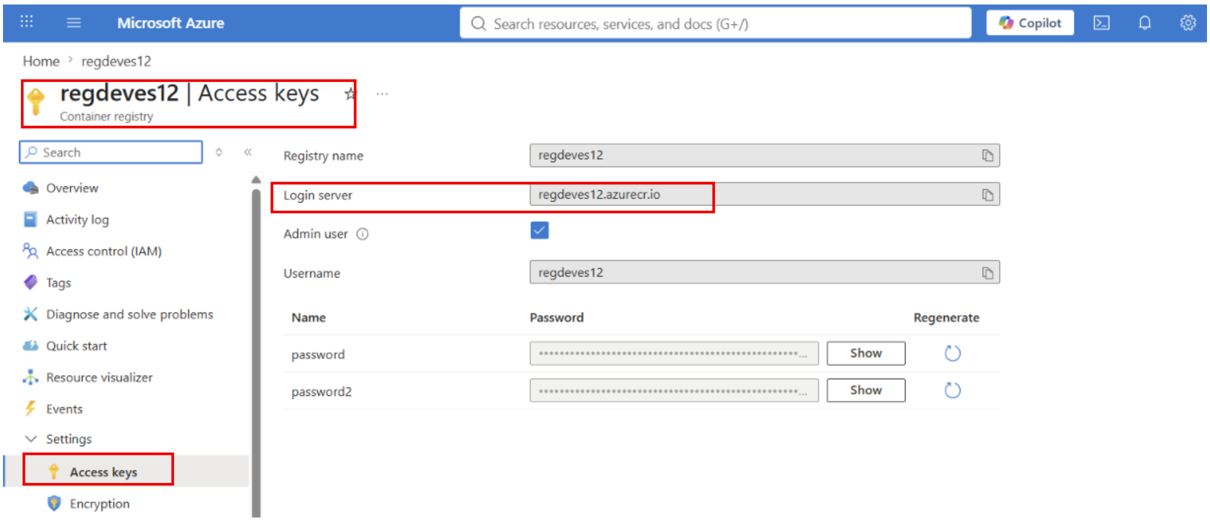
- Open the **powershell.**

docker pull mysql:8.4



Step 2: Tag the Image for Azure Container Registry

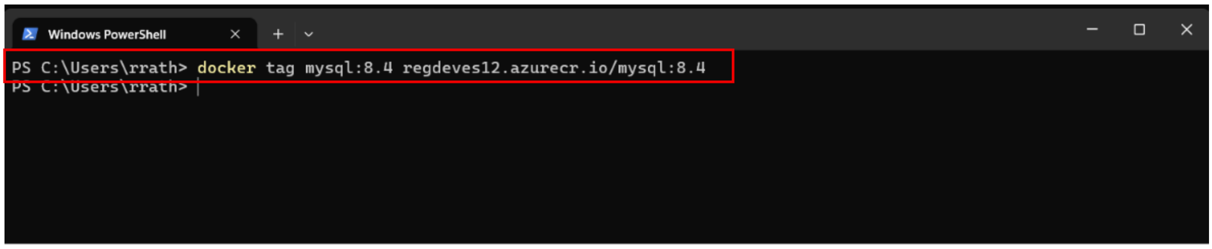
- Go to Azure **container Registry.**
- Navigate to **Access keys.**
- Copy the **Login server.**



docker tag mysql:8.4 <ACR-Login-Server>/mysql:8.4

Example:

docker tag mysql:8.4 **regdeves12.azurecr.io**/mysql:8.4

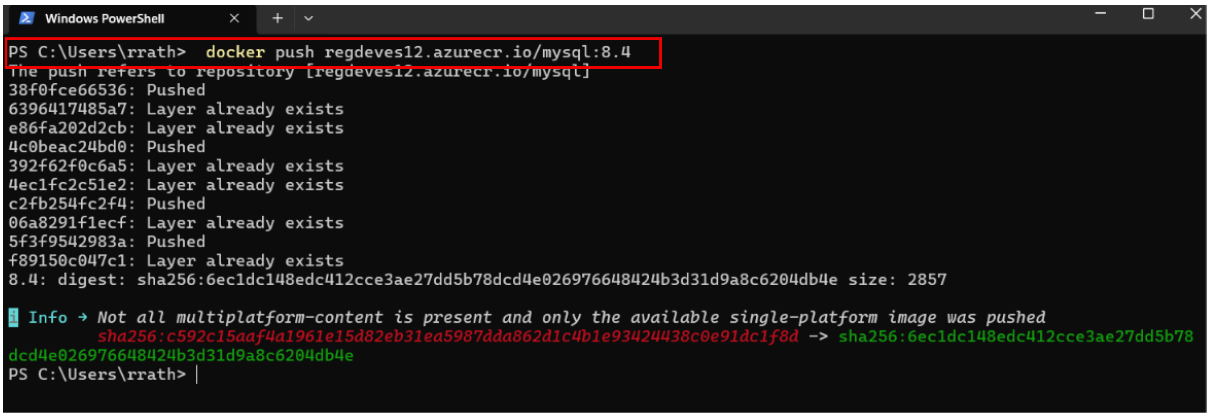


Step 3: Push Image to Azure Container Registry

docker push <ACR-Login-Server>/mysql:8.4

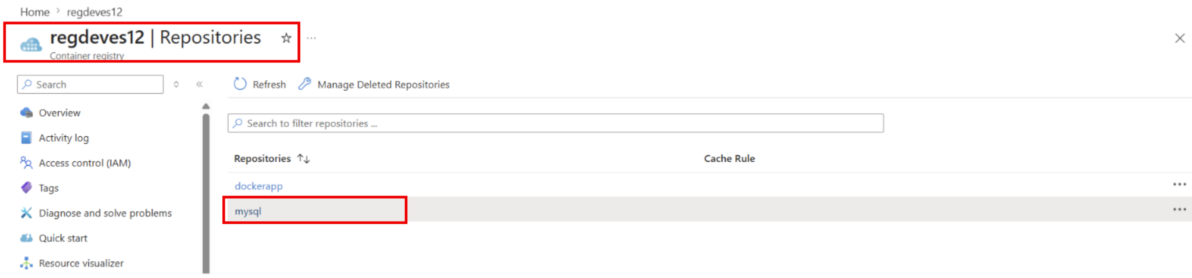
Example:

docker push regdeves12.azurecr.io/mysql:8.4



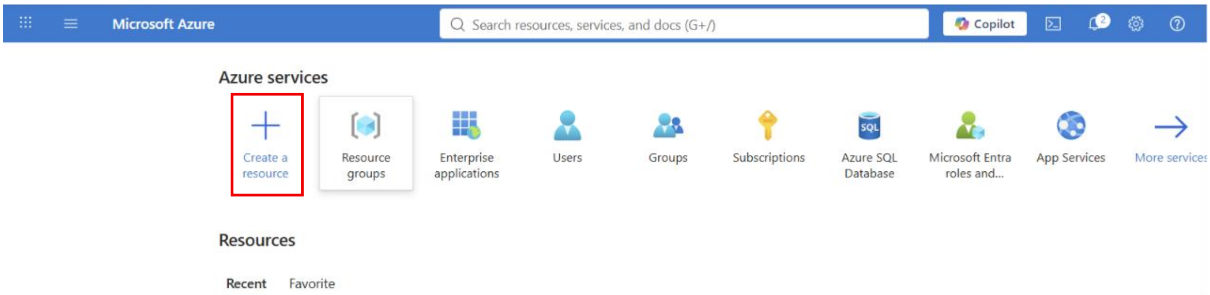
Step 4: Verify Image in Azure Container Registry

- Open Azure **Container Registry**.
- Select **Repositories**.
- Verify that the **mysql:8.4 image is available**.

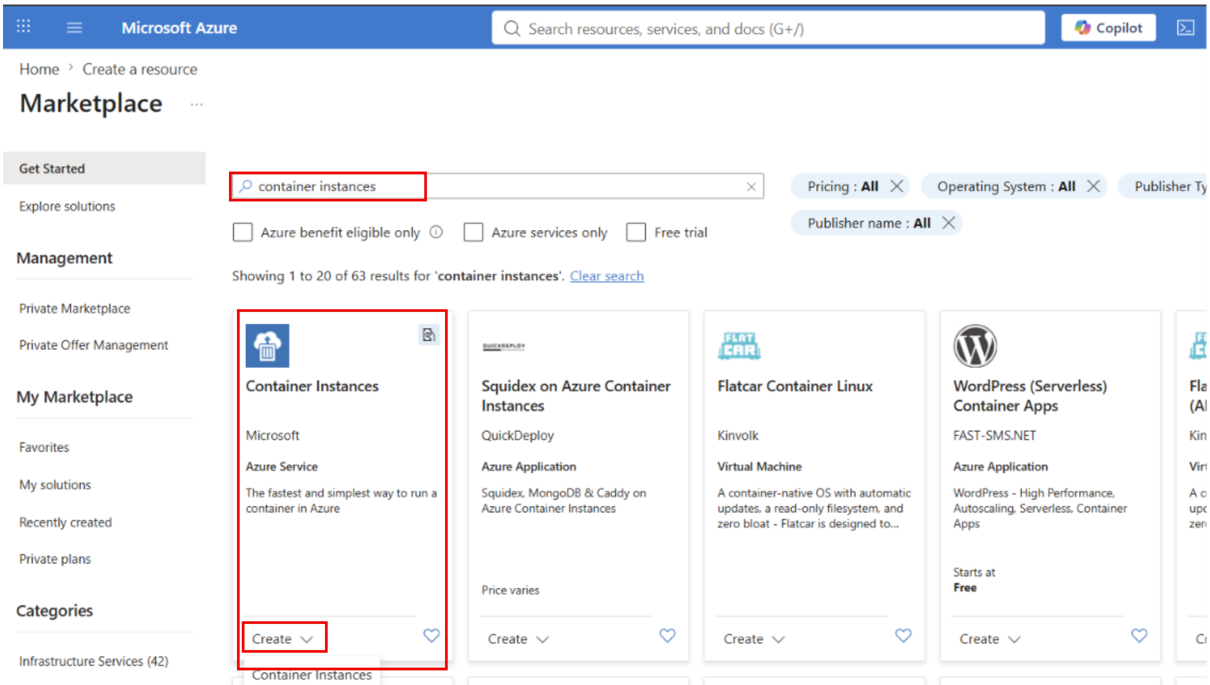


Step 5: Open Azure Container Instances

- Sign in to the Azure Portal.
- Open **Create a Resource**.

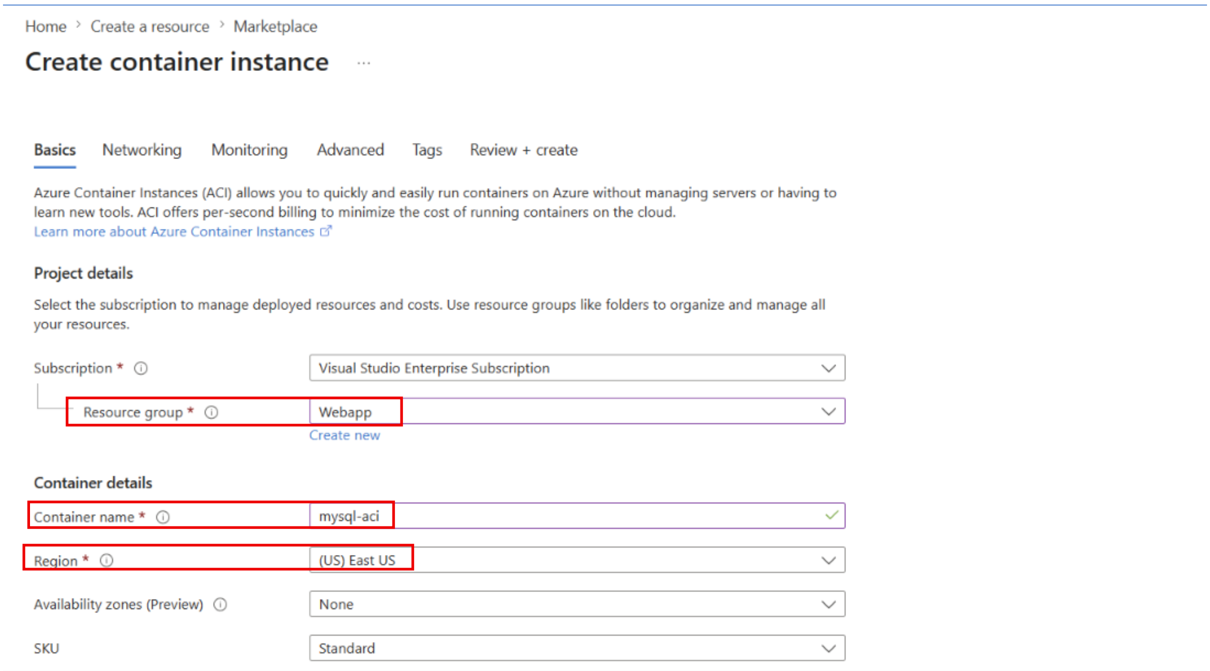


- Search for **Container Instances**.
- Click **Create**.



Step 6: Configure Basic Settings

- Select the existing **Resource Group**.
- Enter a **Container Name**.
- Select the **Region** (Example: East US).

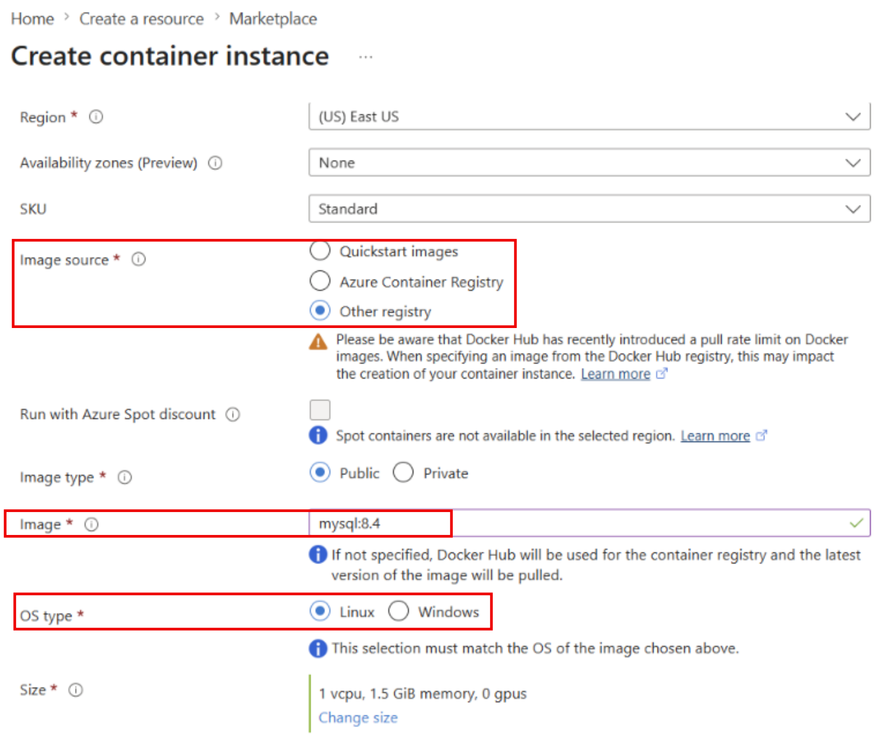


Step 7: Select Docker Hub Image

- Choose **Image Source → Other Registry**.
- Specify the image:

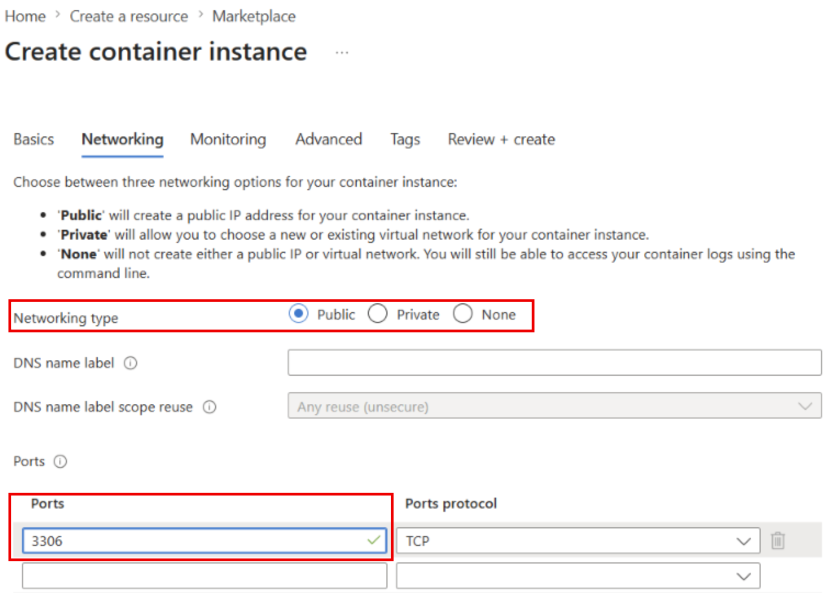
mysql:8.4

- Keep **OS Type** as **Linux**.



Step 8: Configure Networking

- Enable **Public IP** (for the lab).
- Expose **TCP Port 3306** (default MySQL port).

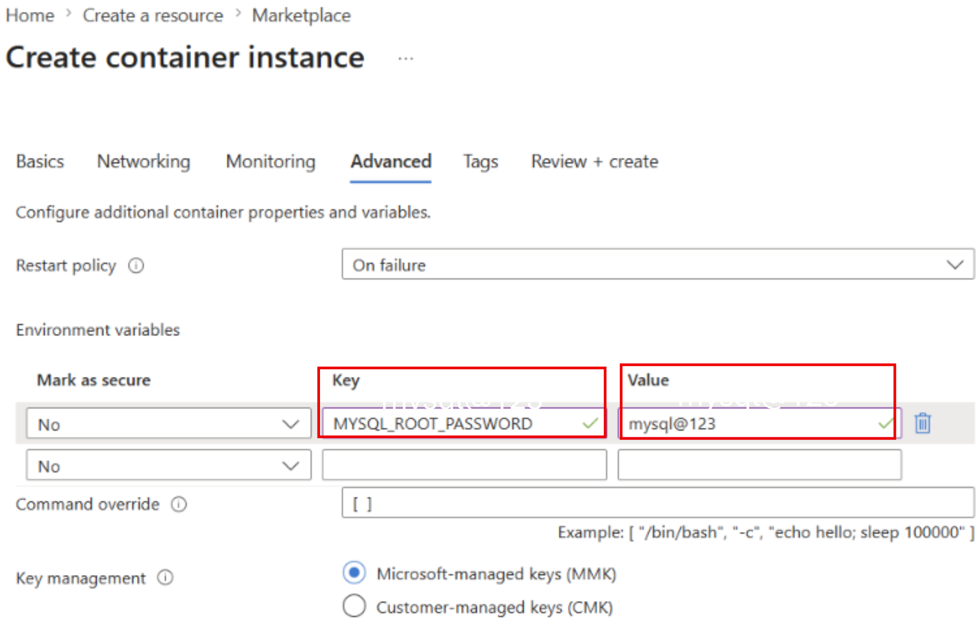


Step 9: Configure Environment Variable

Add the following environment variable:

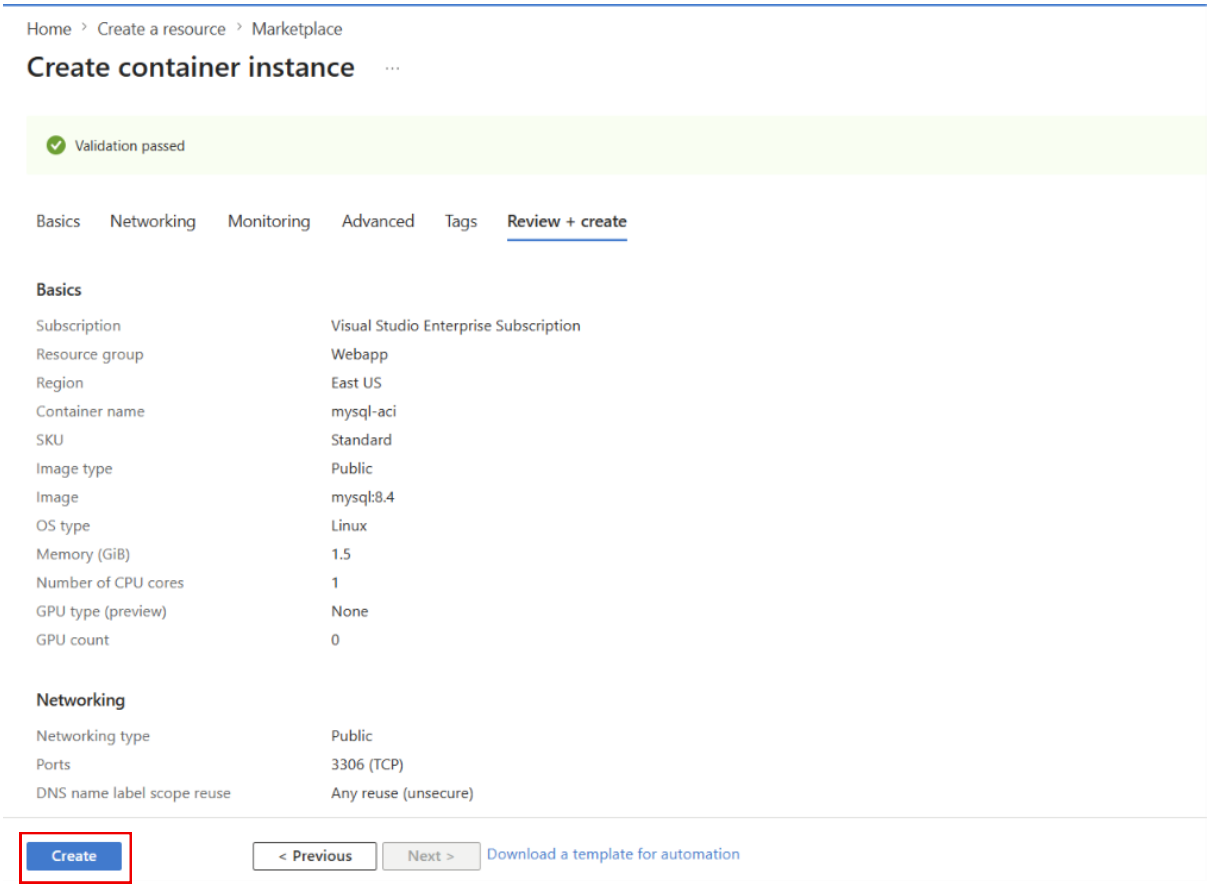
Variable	**Value**
MYSQL_ROOT_PASSWORD	**mysql@123**

- Mark the password as **Secure**.

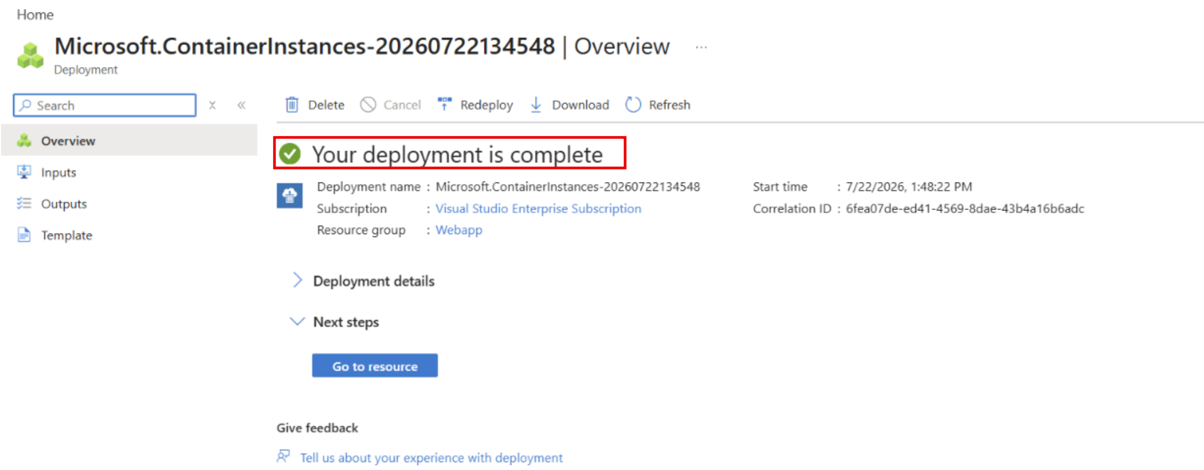


Step 10: Deploy the Container

- Click **Review + Create**.
- Click **Create**.

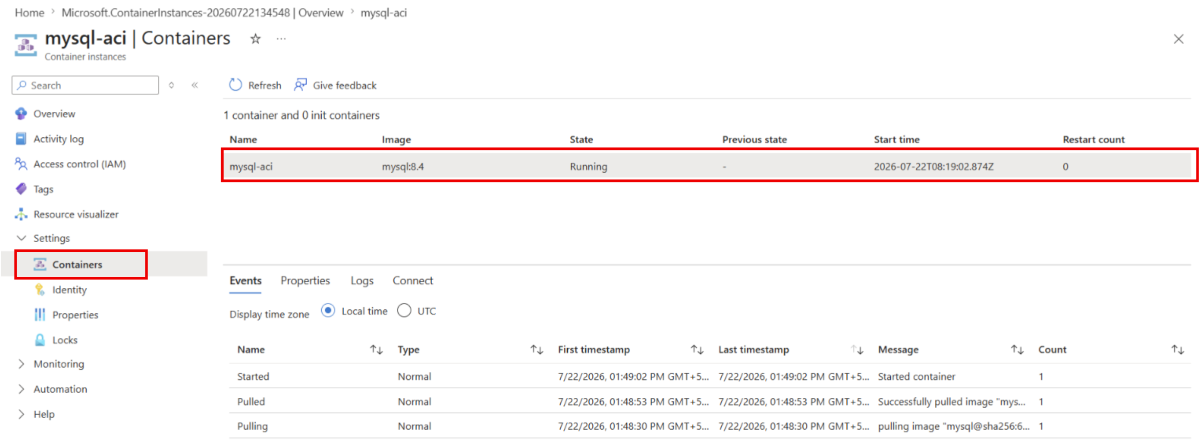


- **Wait until deployment completes.**



Step 11: Verify Container Status

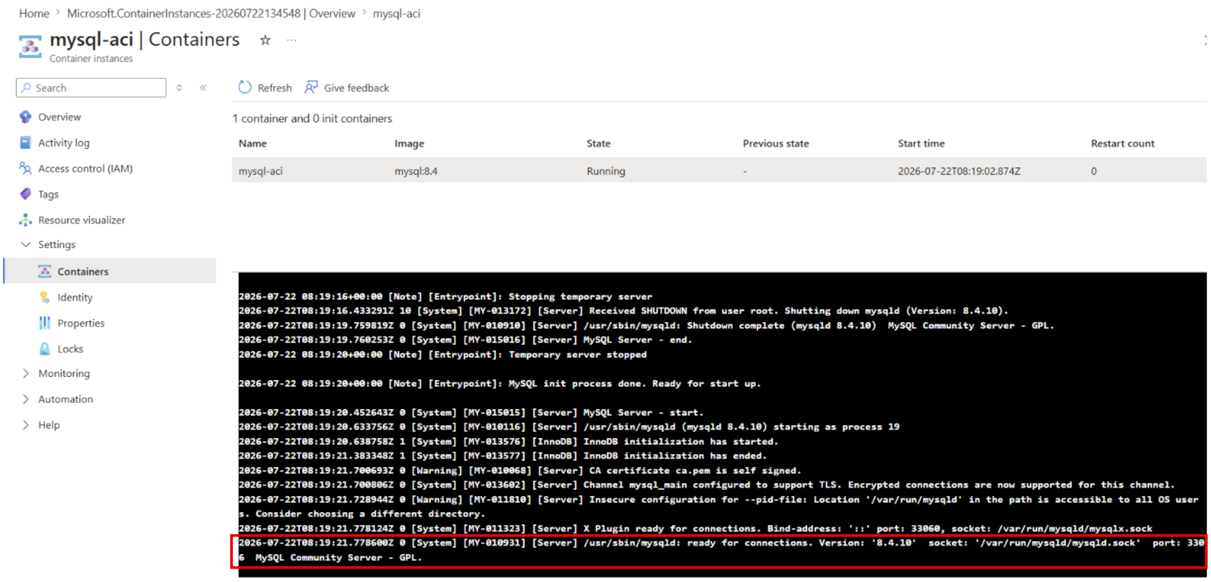
- Open the deployed **Container Instance**.
- Navigate to **Containers**.
- Verify the container state is: Running



Step 12: View Container Logs

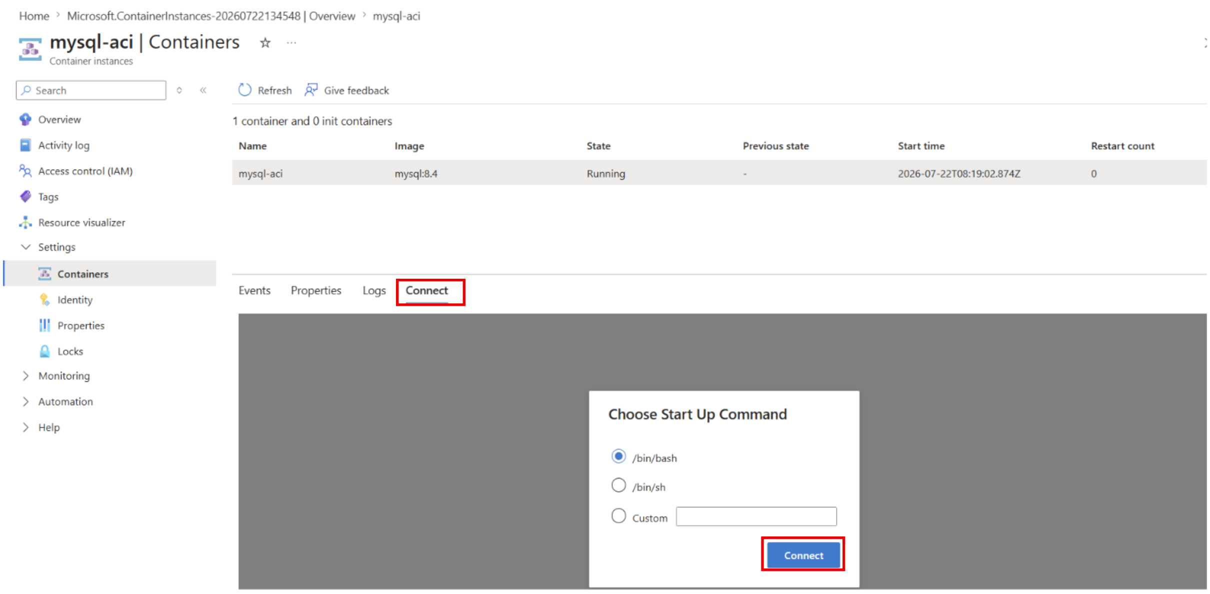
- Open **Containers → Logs**.
- Verify the message:
- MySQL daemon is ready for connections.

This confirms that the MySQL server has started successfully.



Step 13: Connect to the MySQL Container

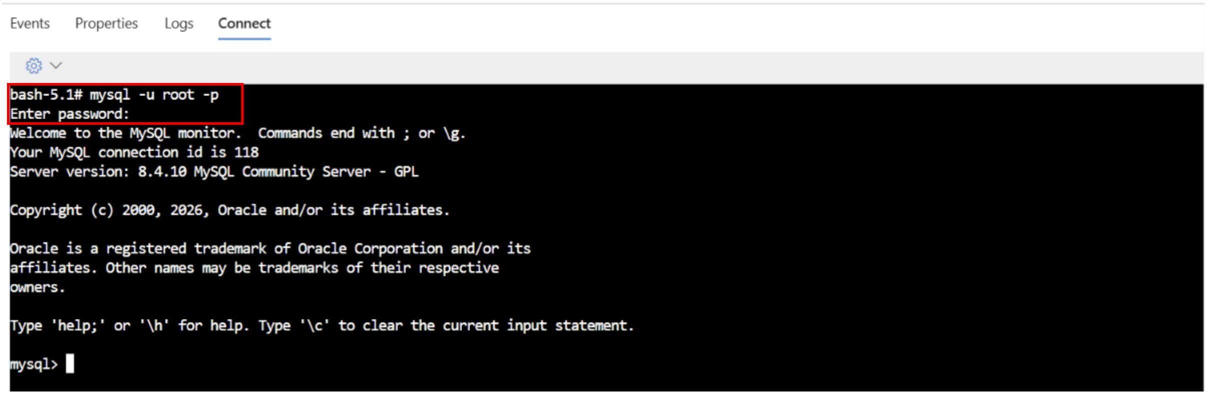
- Open **Containers → Connect**.
- Click on the **Connect.**



- Start the MySQL client:

mysql -u root -p

- Enter the **root password**.

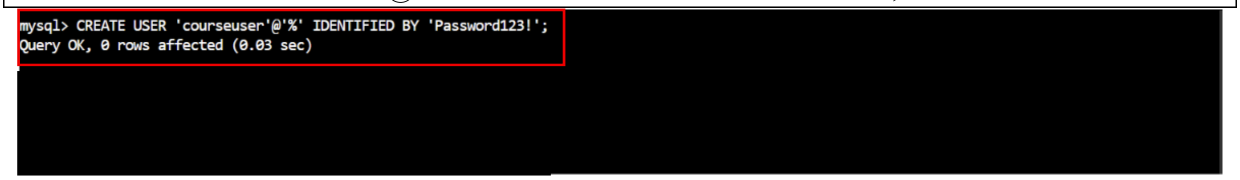


Step 14: Create a Database

CREATE DATABASE CourseDB;

Step 15: Create a User

CREATE USER 'courseuser'@'%' IDENTIFIED BY 'Password123!';



Step 16: Grant Permissions

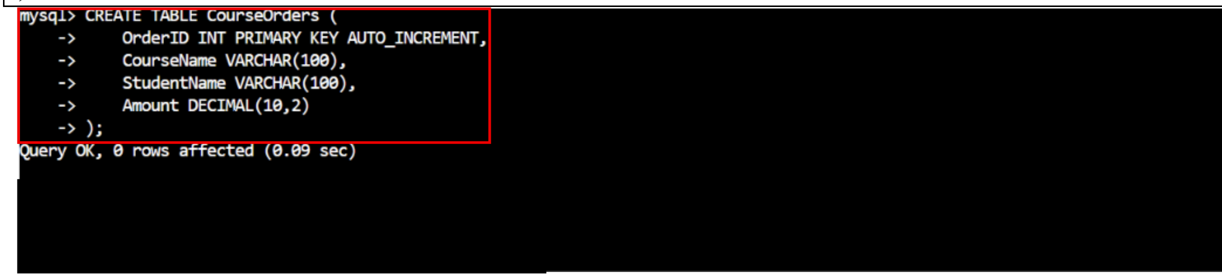
GRANT ALL PRIVILEGES ON CourseDB.* TO 'courseuser'@'%'; FLUSH PRIVILEGES;

Step 17: Use the Database

```
USE CourseDB;
```

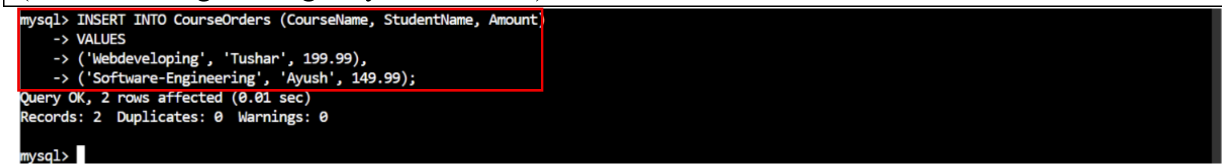
Step 18: Create a Table

```
CREATE TABLE CourseOrders (
    OrderID INT PRIMARY KEY AUTO_INCREMENT,
    CourseName VARCHAR(100),
    StudentName VARCHAR(100),
    Amount DECIMAL(10,2)
);
```



Step 19: Insert Data

```
INSERT INTO CourseOrders (CourseName, StudentName, Amount)
VALUES
('Webdeveloping', 'Tushar', 199.99),
('Software-Engineering', 'Ayush', 149.99);
```



Step 20: View Data

```
SELECT * FROM CourseOrders;
```